

QUESTIONS 8 / 8 completed

Logger

LOGGER

You are working on a project that requires the use of a logging framework. Write a Java program to create an interface called "Logger" with methods for logging different types of messages, such as "debug", "info", "warning", and "error". Implement the interface in a class called "Log4jLogger" that uses the Log4j logging framework.

PROGRAM EDITOR

Java

Run

Save

```
1 interface Logger{
2     void debug(String message);
3     void info(String message);
4     void warning(String message);
5     void error (String message);
6 }
7 class Log4jLogger implements Logger{
8     @Override
9     public void debug(String message){
10         system.out.println("[DEBUG]+"message);
11     }
12     @Override
13     public void info(String message){
14         system.out.println("[INFO]+"message);
15     }
16     @Override
17     public void error(String message){
18         system.out.println("[WARNING]+"message);
19     }
20     @Override
21     public void error(String message){
```

INPUT

Your INPUT goes
here!

QUESTIONS 8 / 8 completed

Character

CHARACTER

You are developing a game that involves different types of characters, such as heroes, villains, and monsters. Write a Java program to create an interface called "Character" with methods for moving, attacking, and taking damage. Implement the interface in classes for each type of character, such as "Hero", "Villain", and "Monster".

PROGRAM EDITOR

Java

Run

Save

```
1 interface Character {
2     void move();
3     void attack();
4     void takeDamage(int d);
5 }
6
7 class Hero implements Character {
8     public void move() { System.out.println("Hero moves"); }
9     public void attack() { System.out.println("Hero attacks"); }
10    public void takeDamage(int d) { System.out.println("Hero takes " + d + " damage"); }
11 }
12
13 class Villain implements Character {
14     public void move() { System.out.println("Villain moves"); }
15     public void attack() { System.out.println("Villain attacks"); }
16     public void takeDamage(int d) { System.out.println("Villain takes " + d + " damage"); }
17 }
18
19 class Monster implements Character {
20     public void move() { System.out.println("Monster moves"); }
21 }
```

INPUT

Your INPUT goes here!

QUESTIONS 8 / 8 completed

Sort

SORT

Write a Java program to create a list of integers and sort them in ascending order using the Collections framework.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.*;
2
3 class main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         List<Integer> list = new ArrayList<>();
7
8         while (sc.hasNextInt()) {
9             list.add(sc.nextInt());
10        }
11
12        Collections.sort(list);
13
14        System.out.println(list);
15    }
16 }
```

INPUT

Your INPUT goes here!

OUTPUT

QUESTIONS 8 / 8 completed

HashSet

HASHSET

Write a Java program to create a HashSet of strings and remove a specific element from it.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.HashSet;
2 import java.util.Scanner;
3
4 class main {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         HashSet<String> set = new HashSet<>();
8
9         System.out.println("Enter element to remove:");
10        String toRemove = sc.hasNext() ? sc.next() : "";
11
12        set.add("Apple");
13        set.add("Banana");
14        set.add("Cherry");
15
16        System.out.println("Original HashSet: " + set);
17
18        if (set.remove(toRemove)) {
19            System.out.println("Removed element: " + toRemove);
20        } else {
21            System.out.println("Element not found.");
```

INPUT

Your INPUT goes here!

QUESTIONS 8 / 8 completed

TreeMap

TREETMAP

Write a Java program to create a TreeMap of strings and integers and iterate over its entries.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.Map;
2 import java.util.TreeMap;
3
4 class main {
5     public static void main(String[] args) {
6
7         TreeMap<String, Integer> map = new TreeMap<>();
8
9         // Add some entries
10        map.put("Apple", 10);
11        map.put("Banana", 20);
12        map.put("Cherry", 30);
13        for (Map.Entry<String, Integer> entry : map.entrySet()) {
14            System.out.println(entry.getKey() + " : " + entry.getValue());
15        }
16    }
17 }
```

INPUT

Your INPUT goes here!

OUTPUT

QUESTIONS 8 / 8 completed

PriorityQueue

PRIORITYQUEUE

Write a Java program to create a PriorityQueue of integers and add elements to it.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.PriorityQueue;
2
3 class main {
4     public static void main(String[] args) {
5         PriorityQueue<Integer> pq = new PriorityQueue<>();
6         pq.add(10);
7         pq.add(20);
8         pq.add(15);
9         pq.add(5);
10
11         System.out.println("PriorityQueue elements: " + pq);
12     }
13 }
```

INPUT

Your INPUT goes here!

OUTPUT

QUESTIONS 8 / 8 completed

LinkedHashMap

LINKEDHASHMAP

Write a Java program to create a LinkedHashMap of strings and integers and retrieve the keys in the order they were inserted.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.LinkedHashMap;
2
3 class main {
4     public static void main(String[] args) {
5         LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
6
7         map.put("One", 1);
8         map.put("Two", 2);
9         map.put("Three", 3);
10
11
12         System.out.println("Keys in insertion order: " + map.keySet());
13     }
14 }
```

INPUT

Your INPUT goes here!

OUTPUT

QUESTIONS 8 / 8 completed

Max-Min

MAX-MIN

Write a Java program to create a Set of integers and perform different operations on it, such as finding the maximum and minimum values.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.Collections;
2 import java.util.HashSet;
3 import java.util.Set;
4
5 class main {
6     public static void main(String[] args) {
7         Set<Integer> numbers = new HashSet<>();
8
9         numbers.add(10);
10        numbers.add(25);
11        numbers.add(5);
12        numbers.add(80);
13        numbers.add(42);
14
15        System.out.println("Set elements: " + numbers);
16
17        int max = Collections.max(numbers);
18        int min = Collections.min(numbers);
19
20        System.out.println("Maximum value: " + max);
21        System.out.println("Minimum value: " + min);
22    }
23 }
```

INPUT

Your INPUT goes
here!

OUTPUT